Week 16

Single-GPU Project Implementation and Review

Milestone Check; Demonstration; Clarity; Reproducibility

Learning Objectives

- Implement a complete machine learning training workflow using PyTorch on a single GPU.
- Demonstrate, document, and submit evidence for a single-GPU project.
- Critically review code for clarity, documentation, and reproducibility.
- Prepare submission materials for assessment.

Session Agenda

- Quick review; connection to previous weeks.
- Single-GPU ML workflow; implementation highlights.
- Code quality; clarity practices and reproducibility.
- Assessment preparation; project checklist.
- Practical activity; peer code reviews.
- Industry insights; links to professional practice.
- Summary and forward look.

Setting the Stage

- You have learned PyTorch fundamentals; practiced on CPU and single GPU.
- Week 16 milestone; move from development to demonstration and documentation.
- Focus shifts; not just making things "work" but making them clear and reproducible.

What is Required for Assessment?

- Implement and run a PyTorch training script on a single GPU.
- Provide clear code; meaningful comments; required outputs (logs or plots).
- Prepare evidence; screenshots or notebook outputs.
- Self-check for clarity, structure, and reproducibility.

Review; Previous Weeks in Context

- Week 13; You moved PyTorch models from CPU to GPU for faster training.
- Week 14; Set up and troubleshot cloud infrastructure for GPU workloads.
- Week 15; Explored basics of distributed training and job management.
- Now; Demonstrate robust single-GPU workflows before scaling up.

Full ML Training Workflow; Single GPU Steps

- Data loading and pre-processing; use appropriate PyTorch data utilities.
- Model definition; leverage nn.Module for modular code.
- Training loop; include clear metric reporting and GPU moves with .to('cuda').
- Save model outputs; checkpoints and logs for assessment submission.

Code Clarity; Industry Standards

- Use clear variable names and modular function design.
- Include inline comments; each major step must be briefly explained.
- Consistent use of versioning and documentation.
- Keep output repeatable; random seeds set, dependencies listed.

Example; Well-Documented PyTorch Function

```
def train_one_epoch(model, dataloader, optimizer, criterion, device):
    model.train()   # Set model to training mode
    running_loss = 0.0
    for inputs, targets in dataloader:
        inputs, targets = inputs.to(device), targets.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    return running_loss / len(dataloader)
```

- Comments explain purpose and key operations.
- Device placement and batch processing handled explicitly.

Checklist; Is Your Project Ready?

- End-to-end script or notebook runs on a single GPU, from data load to evaluation.
- All critical steps have clear, concise code comments.
- Results are saved or displayed as required in task instructions.
- Peer review or self-assessment performed before submission.

Activity; Peer Code Review & Self-Assessment

- Swap scripts with a classmate or use self-assessment checklist.
- Check for; clarity and understandability, usage of device (CPU vs GPU), reproducibility (random seeds, requirements).
- Provide one constructive suggestion for improvement.

Industry Practice Spotlight

- In ML engineering, reproducible results and readable code are essential for teamwork.
- Many companies use code review; CI pipelines; and automated testing before deploying models.
- Documentation and reproducible environments are critical for scaling up to HPC or cloud GPU clusters.

Summary and Next Steps

- Completing a robust, well-documented single-GPU PyTorch project prepares you for distributed and production-level ML.
- Submit your project portfolio and evidence according to instructions.
- Next week: Advance to distributed training and GPU scaling with torchrun and accelerate.